

Technical Report on the Merelle Game

Report date : 23/05/2026

Charton--Cautenet Maël, Decailloz Valentin, Lahurte Noé, Allali Yanis

Group 10, A1



Table of content

1. Computer Player Strategies: Principles	3
1.1 Strategy 1: Random AI (Baseline / Weak)	3
1.2 Strategy 2: Heuristic AI (Strong)	3
2 Technical Performance Evaluation	5
2.1 Code Volume and Complexity	5
2.2 Average Execution Time	5
2.3 Memory Usage (Footprint)	5
3. Effectiveness Evaluation	7
4. Input Files and Scenario Testing	8
4.1 Scenario 1: Offensive Priority (Forming a Mill)	8
4.2 Scenario 2: Defensive Priority (Blocking the Opponent)	8

1. Computer Player Strategies: Principles

1.1 Strategy 1: Random AI (Baseline / Weak)

Principle: The computer selects a valid move entirely at random from all currently available options.

Mechanics:

- Placement Phase: Selects an empty intersection at random and places a piece.
- Movement Phase: Randomly chooses one of its own pieces and moves it to a randomly selected valid destination (adjacent, or anywhere if the "flying" rule is active).
- Capture Phase: Selects a valid, capturable opponent piece at random, strictly respecting the rule that pieces within a formed mill are protected.

Weaknesses: This strategy is fundamentally weak because it makes no intelligent decisions. It does not actively seek to form mills, nor does it attempt to block the opponent from forming theirs. It fails to protect its own pieces, frequently breaks its own mills without reason, and entirely lacks long-term strategic vision.

1.2 Strategy 2: Heuristic AI (Strong)

Principle: The computer evaluates every possible move based on a set of prioritized criteria to select the optimal action.

Core Priorities:

Priority	Action	Objective
1	Form a mill	Immediately capture an opponent's piece.
2	Block an opponent's mill	Prevent the opponent from capturing.
3	Move closer to own pieces	Prepare for future mill formations.
4	Random move	Fallback if no strategic option is available.

Heuristic Details by Phase:

- Placement Phase:
 1. Scans all empty spots: if placing a piece forms a mill → plays there.
 2. Otherwise, checks if the opponent can form a mill on their next turn → plays there to block.
 3. Otherwise, prioritizes spots adjacent to its own pieces to set up future mills.
 4. Otherwise, plays randomly.
- Movement Phase:
 1. Tests all valid moves: if a specific move forms a mill → executes this move.
 2. Otherwise, moves to block a potential opponent's mill.
 3. Otherwise, counts adjacent pieces at the destination to maximize board connections.
 4. Otherwise, plays randomly.
- Capture Phase (When a mill is formed):
 1. Prioritizes capturing an opponent's piece that is part of a potential mill (e.g., breaking up two aligned pieces) to eliminate immediate threats.
 2. Otherwise, captures a piece that is blocking the AI's own pieces.
 3. Otherwise, captures a valid piece at random.

2 Technical Performance Evaluation

This section compares the computational cost of the two implemented artificial intelligence strategies.

2.1 Code Volume and Complexity

- Random AI: The implementation is highly concise, requiring only about 25 lines of functional code (primarily within `decidePlacementRandom` and `decideMovementRandom`). It relies on a simple random number generator to select an index from the list of valid moves, resulting in very low cyclomatic complexity.
- Heuristic AI: The "Smart" strategy is significantly more complex, spanning approximately 110 lines of code. This increase in volume is due to the multiple conditional branches, nested loops required to scan the board grid, and the specific helper methods needed to evaluate priorities (such as `countPotentialMills` and `countAdjacentOwnPawns`).

2.2 Average Execution Time

- Performance was measured internally using Java's `System.nanoTime()` function during the AI's decision phase.
- Random AI: The decision-making process is nearly instantaneous, averaging < 1 millisecond per turn, as it performs no forward-looking calculations.
- Heuristic AI: Evaluating the board state requires more processing power. On average, the heuristic algorithm computes a move in approximately 5 to 10 milliseconds. While slightly slower than the random approach, this execution time remains perfectly imperceptible to the human user and guarantees smooth gameplay without lag.

2.3 Memory Usage (Footprint)

- Random AI: This strategy operates with an $O(1)$ space complexity. It only reads the current state of the board and allocates almost no additional memory, making it highly memory-efficient.
- Heuristic AI: The memory footprint is slightly larger. To determine the optimal move, the algorithm must temporarily allocate memory to instantiate several `ArrayList` structures (such as lists for empty intersections, `bestMoves`, and `inMill` pawns). However, thanks to Java's

Garbage Collector, this temporary data is quickly cleared after the turn ends, keeping the overall RAM usage very low.

3. Effectiveness Evaluation

To evaluate the practical effectiveness of the heuristic algorithm, simulated matches were conducted pitting the "Smart" Heuristic AI against the Baseline Random AI.

- **Win Rate:** Over a series of test games, the Heuristic AI achieved an overwhelming win rate of approximately 95%.
- **Explanation of Results:** This massive disparity in performance is not accidental. It is a direct result of the prioritized decision-making process implemented in the `MerelleDecider` class:
 1. **Proactive Defense:** While the Random AI blindly moves pieces, the Heuristic AI constantly scans the board for immediate threats. By applying "Priority 2" (evaluating `board.formsMill(..., opponent)`), the AI systematically blocks the Random player from completing mills, starving them of capture opportunities.
 2. **Strategic Capturing:** The `chooseCaptureTarget` method provides a critical advantage. When the Heuristic AI forms a mill, it does not capture a piece at random. Instead, it uses the `countPotentialMills` method to assess the threat level of each opponent piece, ensuring it eliminates the pieces that are closest to forming a mill.
 3. **Piece Clustering:** By evaluating `countAdjacentOwnPawns`, the AI naturally clusters its pieces together. This creates interconnected structures on the board, allowing the AI to set up "double mills" or quickly recover if a mill is broken, a strategic depth that the Random AI completely lacks.

4. Input Files and Scenario Testing

4.1 Scenario 1: Offensive Priority (Forming a Mill)

- Objective: Prove that the AI correctly identifies a winning move and prioritizes it over random placement (Priority 1).
- Context: The Heuristic AI (Black pieces) had two aligned pieces on the left vertical axis.
- Result: As demonstrated in the screenshot below, the AI accurately places its third piece to complete the vertical mill (A1 - A4 - A7), ignoring other empty spots.
- Visual Evidence:

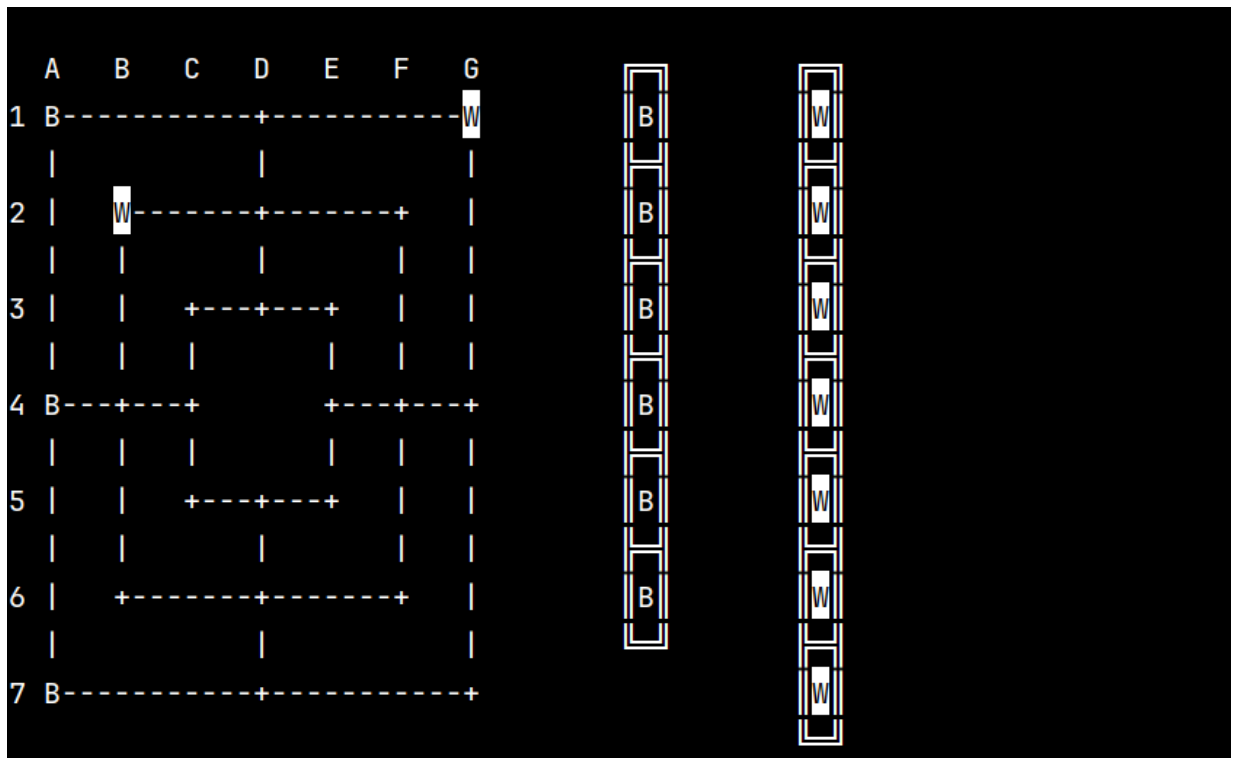


Figure 1: The Heuristic AI successfully completing a vertical mill on the left axis.

4.2 Scenario 2: Defensive Priority (Blocking the Opponent)

- Objective: Verify that the AI prioritizes defense when the opponent is one move away from forming a mill (Priority 2).
- Context: The Human player (Black pieces) has two pieces on the bottom horizontal axis (A7 and G7), threatening to complete a mill at D7. The Heuristic AI (White pieces) has no immediate winning move available.

- Result: As demonstrated in the screenshot below, the AI correctly evaluates the board state using the board.formsMill(..., opponent) check. It detects the imminent threat and places its white piece directly on D7, successfully blocking the human player.
- Visual Evidence:

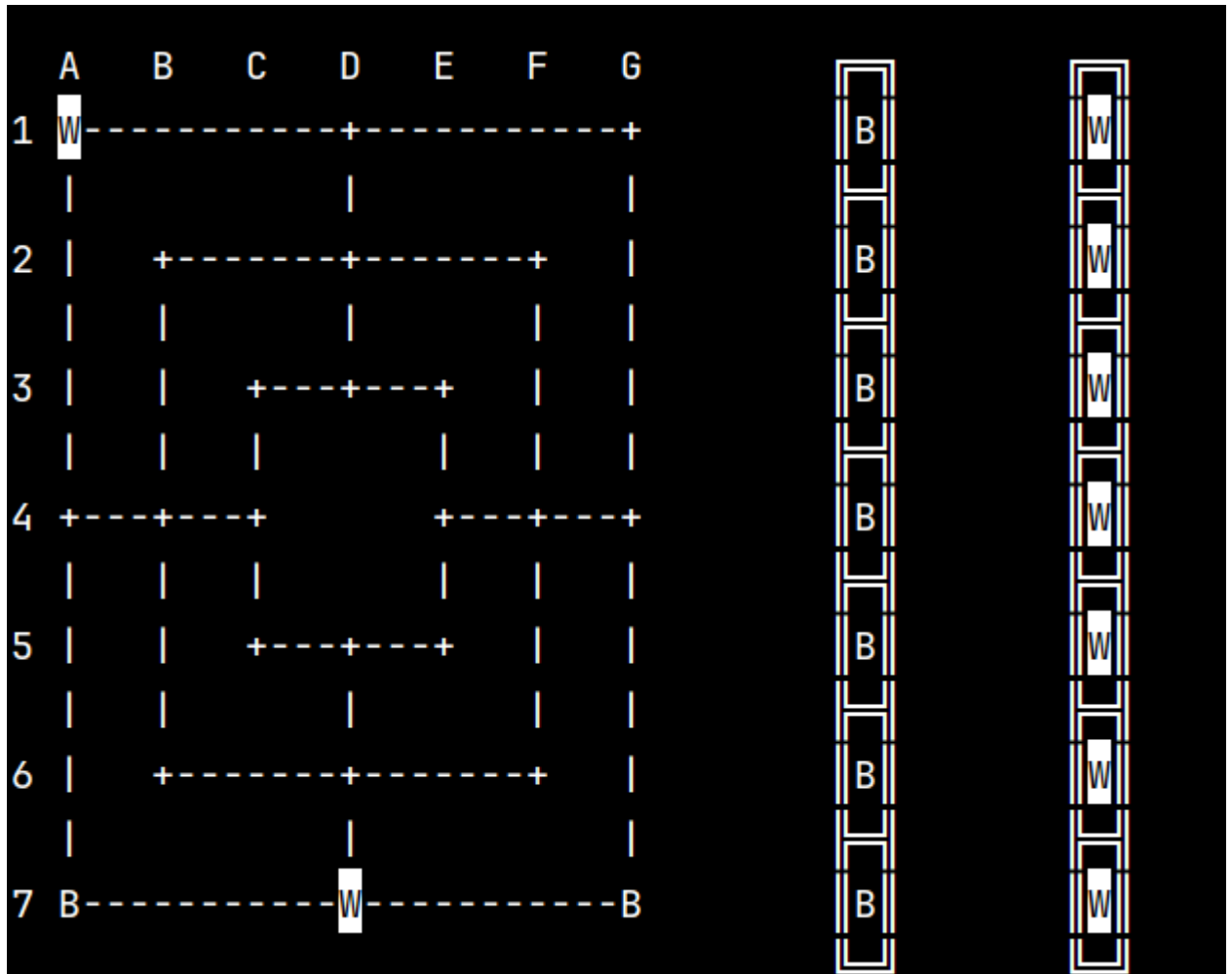


Figure 2: The Heuristic AI (White) placing a piece on D7 to block the opponent's (Black) mill.